

This homework has 9 questions, for a total of 100 points and 10 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up or any AI tools*.

(0 pts) 0. **Before you start; before you submit.**

- (a) Before starting this assignment, carefully read [Handout 1](#) (Running Times and Asymptotics) and the “Good Writing” section of [Handout 2](#) (the induction section is optional but recommended), and apply them to your work.
- (b) If applicable, state the name(s) and username(s) of your collaborator(s).

Solution: No collaborator

1. **L^AT_EX (optional extra credit).**

We (along with the rest of the math-writing world) use L^AT_EX to write all homeworks and exams. Throughout the semester, we strongly encourage you to write all of your homework solutions in L^AT_EX too. This extra credit problem will give you some practice with L^AT_EX.

To compile L^AT_EX code, you can use the website overleaf.com, or download a L^AT_EX editor of your choice. To compile each of the EECS 376 homework assignments in L^AT_EX you will also need to upload the provided `header.tex` file into your project.

L^AT_EX is *not* a “what you see is what you get” word-processing software like you may be accustomed to (Word, Google Docs, etc.). Instead, a L^AT_EX document is a *program* that describes the desired output, and is *compiled* into a human-readable format.

Here are some good resources about L^AT_EX practices:

- [Writing Beautifully in L^AT_EX](#),
- [Short Math Guide for L^AT_EX](#),
- [DOs and DON'Ts when typesetting a document](#).

To receive the extra credit points, you must typeset this **entire** assignment in L^AT_EX, and typeset the two snippets of text given in parts (a) and (b) in L^AT_EX. **Put your solution to this problem in the Verbatim environment as follows so that we can see the L^AT_EX source code in your compiled PDF** (do not use the Verbatim environment for the rest of the assignment):

```
This text is in the \texttt{Verbatim} environment. \emph{This} word is
italized. Here is some math: $1+1=2$.
```

Here are some examples of L^AT_EX syntax you may need:

- In-line math
- Less-than-or-equal-to symbol
- Quote marks
- Bulleted lists

- Display math
- The algorithmic environment

You may copy and modify the source code of this document (e.g. part (b) asks you to write an algorithm in $\text{\LaTeX}$, and other problems in this document already contain algorithms). You may also use the Internet to search for $\text{\LaTeX}$ syntax.

(5 EC pts)

- (a) The following snippet is about some key features of the big-O notation extracted from [Handout 1](#).

For (non-negative) functions $f(n)$ and $g(n)$, we say “ $f(n) = O(g(n))$ ” if there exist positive constants c, n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. The key features here are:

- “there exists a positive constant $c \dots f(n) \leq c \cdot g(n)$ ”: this says that $f(n)$ is *upper bounded* by some *constant multiple* of $g(n)$. That is, O -notation “hides,” or ignores, constant factors. Note also that it captures only an *upper bound*: $f(n)$ might actually be *much smaller* than $c \cdot g(n)$, or not. A big-O bound by itself says nothing about which is the case.
- “there exists a positive constant $n_0 \dots$ for all $n \geq n_0$ ”: this says that the upper bound holds for all n *above some constant “threshold.”* However, it says nothing about the relationship between $f(n)$ and $g(n)$ *below* the threshold; it could be that $f(n)$ greatly exceeds (even a huge multiple of) $g(n)$ in that range. Moreover, the threshold n_0 can be *any constant*—it could be one, or ten, or a billion, or a googolplex—and the notation hides its exact value.

Solution:

For (non-negative) functions $f(n)$ and $g(n)$, we say “ $f(n) = O(g(n))$ ” if there exist positive constants c, n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

The key features here are:

```
\begin{itemize}
  \item “there exists a positive constant  $c$  ...  $f(n) \leq c \cdot g(n)$ ”:
    this says that  $f(n)$  is \textit{upper bounded} by some constant
    multiple of  $g(n)$ .
    That is,  $O$ -notation “hides,” or ignores, constant factors.
    Note also that it captures only an \textit{upper bound} on  $f(n)$ ,
    which might actually be
    \textit{much smaller} than  $c \cdot g(n)$ , or not.
    A Big-O bound by itself says nothing about which is the case.

  \item “there exists a positive constant  $n_0$  ... for all  $n \geq n_0$ ”:
    this says that the upper bound holds for all  $n$  above some
    constant “\textit{threshold}.”
```

However, it says nothing about the relationship between $f(n)$ and $g(n)$ below the threshold;
it could be that $f(n)$ greatly exceeds (even a huge multiple of) $g(n)$ in that range.
Moreover, the threshold n_0 can be any constant: it could be one, or ten, or a billion, or a googolplex---and the notation hides its exact value.

- (5 EC pts) (b) The following snippet is about [Fibonacci numbers](#).

The Fibonacci numbers may be defined by the recurrence relation

$$F_n = F_{n-1} + F_{n-2}$$

with base cases $F_0 = 0$ and $F_1 = 1$. We can compute the n -th Fibonacci number using the following recursive algorithm:

Input: a natural number n
Output: the n -th Fibonacci number

```

1: function FIB( $n$ )
2:   if  $n = 0$  then
3:     return 0
4:   if  $n = 1$  then
5:     return 1
6:   return FIB( $n - 1$ ) + FIB( $n - 2$ )

```

Solution:

The Fibonacci numbers may be defined by the recurrence relation

```

\[
F_n = F_{n-1} + F_{n-2}
\]

```

with base cases $F_0 = 0$ and $F_1 = 1$. We can compute the n -th Fibonacci number using the following recursive algorithm:

```

\begin{algorithm}
\caption{Recursive Fibonacci}
\begin{algorithmic}
\Require A natural number  $n$ 

```

```
\Ensure The $n$-th Fibonacci number
\Function{Fib}{$n$}
  \If{$n = 0$}
    \State \Return $0$
  \EndIf
  \If{$n = 1$}
    \State \Return $1$
  \EndIf
  \State \Return $\text{\Call{Fib}{n-1}} + \text{\Call{Fib}{n-2}}$
\EndFunction
\end{algorithmic}
\end{algorithm}
```

2. Properties of logarithms.

The objective of this problem is to recall some important properties of logarithms that we'll use throughout the course. First recall the following properties of exponentiation for any real numbers b, x, y , assuming that $b \neq 0$:

P1: $(b^x)^y = b^{xy}$

P2: $b^x = b^y$ if and only if $x = y$, additionally assuming that $b \neq 1$.

Now, for positive real numbers $b \neq 1$ and x , the definition of the base- b logarithm is:

D1: $y = \log_b x$ is the unique real number for which $b^y = x$. That is, it satisfies the identity $b^{\log_b x} = x$.

Combining D1 with P2, we see that $x = y$ if and only if $\log_b x = \log_b y$ (assuming that b, x, y are positive and $b \neq 1$). So, equality is preserved by taking logarithms or exponentials (with the same base) on both sides of an equation, assuming the hypotheses are met. You may use this fact in your solutions.

Prove each of the following properties of logarithms, for any positive real numbers (not necessarily integers) a, b, c, n where $b, c \neq 1$. Cite definition D1 and properties P1 and P2 where you use them; you may also use (with citation) the results of previous parts of the problem. Do not use any other assumptions about, or properties of, logarithms and exponentials.

For reference, we have provided a solution to the first part, which you can use as a model for later solutions.

(0 pts) (a) $\log_b(a^n) = n \log_b a$. (Be careful: n is not necessarily an integer)

Solution:

By P2, the statement we want to prove is equivalent to $b^{\log_b(a^n)} = b^{n \log_b a}$ (since $b \neq 1$). By D1, the left-hand side is equal to a^n , so it suffices to prove that $a^n = b^{n \log_b a}$. Indeed,

$$b^{n \log_b a} = (b^{\log_b a})^n = a^n,$$

where the first equality is by P1, and the second equality is by D1.

(As a remark, this proof does not rely on the assumption that n is positive, so the statement holds even more generally for any real number n . However, later parts require n to be positive.)

- (3 pts) (b) $\log_b a = \frac{\log_c a}{\log_c b}$. (This is sometimes called the “change-of-base theorem,” because it lets us change the base of the logarithm by dividing by an appropriate factor. Note that $\log_c b \neq 0$ because $b \neq 1$, so the division is well defined.)

Solution:

Let $x = \log_b a, y = \log_c a, z = \log_c b$, so it is equivalent to

$$x = \frac{y}{z}$$

By D1, $a = b^x, a = c^y, b = c^z$, so $b^x = c^y$,

so $\log_b b^x = \log_b c^y$, which implies $x = y \log_b c$,

For $b = c^z$, it implies $1 = z \log_b c$, so $\log_b c = \frac{1}{z}$,

Combine these two equations, we get $x = \frac{y}{z}$,

so $\log_b a = \frac{\log_c a}{\log_c b}$.

- (3 pts) (c) $a^{\log_b n} = n^{\log_b a}$. (This is sometimes called the “log-switching theorem” since it says that we can “switch” a and n .)

Solution:

By P2, it is equivalent to $\log_b n = \log_a n^{\log_b a}$,

By (a), $\log_a n^{\log_b a} = \log_b a \log_a n$,

so it is equivalent to $\log_b n = \frac{\log_b n}{\log_a n}$,

Let $x = \log_b n$, By D1, $n = b^x$, which implies $b = n^{\frac{1}{x}}$, so $\log_b n = \frac{1}{\log_n b}$,

so $\log_b a = \frac{\frac{1}{\log_n b}}{\frac{1}{\log_n a}} = \frac{\log_n a}{\log_n b}$, which has proved in (b).

- (3 pts) (d) The ratio $\frac{\log_b n}{\log_c n}$ (for $n \neq 1$) depends only on b and c , not n .

This implies that we can often omit the (constant) base when writing logarithms in asymptotic notation, e.g., we can write $O(\log n)$ instead of $O(\log_2 n)$, because $O(\cdot)$ hides constant factors that do not depend on n . However, we cannot omit the base when the logarithm appears in the exponent, as in $O(3^{\log_2 n})$.

Solution:

By what I have proved in (c),

$$\frac{\log_b n}{\log_c n} = \frac{\log_n c}{\log_n b} = \log_b c,$$

so it depends only on b and c , not n .

3. Welcome to EECS 376!

To express our excitement for the start of EECS 376, we present an algorithm that prints "Welcome to EECS 376!" multiple times, based on input parameter n .

Input: positive integer n represented in binary

```
1: function WELCOME( $n$ )
2:   for  $i = 1, 2, \dots, n$  do
3:     for  $j = 1, 2, \dots, i$  do
4:       PRINT("Welcome to EECS 376!")
```

- (4 pts) (a) Give the *tightest correct asymptotic* (big- O) bound, as a function of n , on the worst-case running time of the WELCOME(n) algorithm.

Solution:

The inner loop executes i times, so the total is $\sum_{i=1}^n i = n(n+1)/2 = \Theta(n^2)$. Thus the tightest big- O bound is $O(n^2)$.

- (4 pts) (b) Let s be the input size of the integer n , i.e., the number of bits needed to represent n in binary. Give the tightest correct asymptotic (big- O) bound, as a function of s , on the worst-case running time of the WELCOME algorithm. State whether or not the algorithm is *efficient*, i.e., runs in at most polynomial time with respect to the input size s .

Solution:

For an s -bit binary integer, $2^{s-1} \leq n < 2^s$. Therefore $\Theta(n^2) = \Theta(4^s)$, which is exponential rather than polynomial in s . So the algorithm is not efficient.

- (4 pts) (c) Does your answer in [Part b](#) change if n is represented in decimal (base-10) instead? That is, let s' be the input size of the integer n , i.e., the number of digits needed to represent n in base-10. Give the tightest correct asymptotic (big- O) bound, as a function of s' , on the worst-case running time of the WELCOME algorithm. State whether or not the algorithm is *efficient*, i.e., runs in at most polynomial time with respect to the input size s' .

Solution:

For an s' -digit decimal integer, $10^{s'-1} \leq n < 10^{s'}$. Hence the running time is $\Theta(100^{s'})$, exponential in s' , so it is not efficient.

- (4 pts) (d) Now consider a different algorithm $\mathcal{A}$ that takes an input an integer n and has worst-case running time $O(\log n)$.

Suppose n is represented in binary and let s be the input size (number of bits in n). Give the tightest correct asymptotic (big- O) bound, as a function of s , on the worst-case running time of algorithm $\mathcal{A}$. State whether or not the algorithm is *efficient*.

Does your answer change if n is represented in base-10 instead? That is, let s' be the input size (number of digits needed to represent n in base-10). Give the tightest correct

asymptotic (big- O) bound, as a function of s' , on the worst-case running time of algorithm $\mathcal{A}$. State whether or not the algorithm is *efficient*.

Solution:

For binary input, $\log n = \Theta(s)$, and for decimal input, $\log n = \Theta(s')$. So $\mathcal{A}$ runs in $O(s)$ and $O(s')$, respectively, and is efficient in both representations.

4. Practice with asymptotics (“big-Oh, big-Omega, big-Theta”).

Let $f(n)$ and $g(n)$ be positive functions. Recall that $f(n) = O(g(n))$ if there exist constants $c, n_0 > 0$ such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

- In words, this says: “ $f(n)$ is *at most* (or: is *upper-bounded by*) a *constant multiple* of $g(n)$, for *all large enough* n .” (“All large enough n ” refers to the clause “all $n \geq n_0$ ”, i.e., all n beyond some fixed threshold.)
- It can be shown that:
 1. If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ is finite, $f(n) = O(g(n))$ is true.
 2. If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ is infinite, $f(n) = O(g(n))$ is false.
 3. If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ does not exist, we cannot conclude whether $f(n) = O(g(n))$ is true or false. For example, for $f(n) = |\sin(n)|$ and $g(n) = 2$ the limit does not exist, but $f(n) = O(g(n))$.

Similarly, $f(n) = \Omega(g(n))$ if there exist constants $c, n_0 > 0$ such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$.

- In words, this says that “ $f(n)$ is *at least* (or: is *lower-bounded by*) a positive-constant multiple of $g(n)$, for all large enough n .”
- It can be shown that:
 1. If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ is greater than zero (including infinity), $f(n) = \Omega(g(n))$ is true.
 2. If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ is zero, $f(n) = \Omega(g(n))$ is false.
 3. If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ does not exist, we cannot conclude whether $f(n) = \Omega(g(n))$ is true or false.

Moreover, it can be seen that $f(n) = \Omega(g(n))$ is equivalent to $g(n) = O(f(n))$, since $f(n) \geq c \cdot g(n)$ if and only if $g(n) \leq (1/c) \cdot f(n)$, and c is a positive constant if and only if $1/c$ is a positive constant.

Finally, $f(n) = \Theta(g(n))$ if both $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

For the following pairs of functions, state, with rigorous proof, whether or not each of the following hold: $f(n) = O(g(n))$, $f(n) = \Omega(g(n))$, $f(n) = \Theta(g(n))$.

Hint: You might find L’Hôpital’s Rule useful, which says (in particular): if $f(n), g(n)$ are differentiable, $g'(n) \neq 0$ for all large enough n , and $\lim_{n \rightarrow \infty} g(n) = \infty$, then $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)}$ whenever the latter limit exists. (Here f' and g' are respectively the derivatives of f and g .) You may use a calculator or computer to calculate derivatives.

- (3 pts) (a) $f(n) = \log_2(n^3)$, $g(n) = \log_5(n^{1/8})$.

Optional ungraded challenge: generalize to similar-looking f and g , beyond specific constant bases and exponents.

Solution:

$$f(n) = 3 \log_2 n \text{ and } g(n) = \frac{1}{8} \log_5 n$$

So $\frac{f(n)}{g(n)} = 24 \log_2 5$, a positive constant.

Therefore both $f = O(g)$ and $f = \Omega(g)$ hold,

so $f(n) = \Theta(g(n))$.

- (4 pts) (b) $f(n) = (376 + \sin(n)) \cdot n$, $g(n) = 203n$.

Optional ungraded challenge: generalize to similar-looking f and g , beyond specific constant coefficients or exponents.

Solution:

Since $-1 \leq \sin n \leq 1$, $375n \leq f(n) \leq 377n$.

With $g(n) = 203n$, the ratio of $\frac{f(n)}{g(n)}$ is bounded above and below by positive constants.

Thus $f = O(g)$ and $f = \Omega(g)$, So $f(n) = \Theta(g(n))$.

- (6 pts) (c) $f(n) = n^3 \cdot 2^{3n}$, $g(n) = 3^{2n}$.

Optional ungraded challenge: generalize to similar-looking f and g , beyond specific constants in the bases and exponents.

Solution:

$$\frac{f(n)}{g(n)} = n^3 (8/9)^n \rightarrow 0, \text{ when } n \rightarrow \infty.$$

Thus $f = O(g)$ is true, while $f = \Omega(g)$ is false;
consequently $f = \Theta(g)$ is false.

5. Prof. Upsilon and their oopsie claims.

Prof. Υ is an absent-minded computer scientist, known for their occasional “oopsie” moments. Recently, Prof. Υ came up with three algorithms with the following running times:

- Algorithm X has worst-case running time $T_X(n) = O(n^2)$
- Algorithm Y has worst-case running time $T_Y(n) = \Theta(n \log n)$
- Algorithm Z has worst-case running time $T_Z(n) = \Theta(n)$

Recall that *worst-case* running time of an Algorithm A means the *maximum* number of steps for which A runs, over all possible inputs of size n .

Now, Prof. Υ has made a series of claims about these algorithms. State, with proof, whether Prof. Υ 's claim is necessarily true, necessarily false, or not necessarily either based on the given information. If it is not necessarily either, give an example when it is true and an example when it is false. For instance, such an example could be of the form: “Suppose Algorithm Y takes time $8n \log n$ for some inputs and time $4n$ for the remaining inputs.”

Solution:

- (4 pts) (a) “Algorithm Y has running time $O(n^2)$.”

Solution:

It is true, for $n \geq 2$, $n \log n \leq n^2$, so $T_Y(n) = \Theta(n \log n) = O(n^2)$.

- (6 pts) (b) “On every sufficiently large input, Algorithm Z runs faster than Algorithm Y .”

Solution:

Not always true.

It is true if every size- n input takes n steps for Z and $n \log n$ for Y .

It is false if, there exist a special input n such that $T_Z = n$ and $T_Y = 1$.

- (6 pts) (c) “For all large enough n , there is an input of size n for which Y runs faster than X .”

Solution:

Not always true.

If $T_X(n) = n^2$ and $T_Y(n) = n \log n$ on every input, it is true.

If $T_X(n) = n$ and $T_Y(n) = n \log n$ on every input, it is false.

- (6 pts) (d) “For all large enough n , there is an input of size n for which Z runs faster than Y .”

Solution:

It is true. For sufficiently large n , choose an input that Y is in worst-case, so $T_Y(n) \geq cn \log n$.

Every input has $T_Z(n) \leq Cn$. Since $c \log n > C$ eventually, this input makes Z faster than Y .

6. Set of proofs by contra-.

Recall that a *set* is a collection of distinct elements. Below are some important set notations:

- **Elements of a set:** $x \in A$ means x is an element of set A .
- **Complement of a set:**¹ $\overline{A}$ refers to the set of all elements in the universal set U that are not in A .
- **Subset:** $A \subseteq B$ means every element of A is also an element of B .
- **Intersection of sets:** $A \cap B$ is the set of elements that are in both A and B .
- **Union of sets:** $A \cup B$ is the set of elements that are in A , in B , or in both.
- **Set difference:** $A \setminus B$ is the set of elements that are in A but not in B . We can also write it as $A \cap \overline{B}$.

- (4 pts) (a) Recall the transitive property of the subset relation:

¹In some texts, you may encounter notations like A^C , A^c , A' , etc. to refer to the complement of A .

If $A \subseteq B$ and $B \subseteq C$, then $A \subseteq C$.

Prove the above property by contradiction.

Solution:

Assume $A \not\subseteq C$. Then some $x \in A$ has $x \notin C$. From $A \subseteq B$ we get $x \in B$, and from $B \subseteq C$ we get $x \in C$, a contradiction. Hence $A \subseteq C$.

- (4 pts) (b) Recall the DeMorgan's Law, which says:

If $x \in \overline{A \cap B}$, then $x \in \overline{A} \cup \overline{B}$.

State the contrapositive of the statement. Then, prove the contrapositive.

Solution:

The contrapositive is $x \notin \overline{A} \cup \overline{B} \Rightarrow x \notin \overline{A \cap B}$. If x is in neither complement, then $x \in A$ and $x \in B$, so $x \in A \cap B$, which is equivalent to $x \notin \overline{A \cap B}$.

7. Requirements for the potential method.

Recall that the potential method requires several properties to hold in order to prove that a given process must eventually halt, within a certain number of "time units." These requirements include (but are not limited to):

1. the potential cannot go below some fixed lower bound; or if it does, the process must immediately halt.
2. the potential must *decrease by (at least) some fixed positive amount* with each time unit.

For each of the following processes and candidate potential functions, state with justification whether they satisfy each of the above properties. If not, either describe an alternative potential function that does satisfy both properties (and allows us to conclude that the process must halt), or briefly explain why the process might not ever halt (and hence there is *no* valid potential function for the process).

- (4 pts) (a) You take a road trip from Michigan to California, which are about 2360 miles apart. On each day, you drive *half* of your remaining distance to California, and then stop to see the sights. So, you drive (roughly) 1180 miles on the first day, 590 miles on the second day, 295 miles on the third day, and so forth. Define a time unit to be one day of driving, and the potential s_i after i days of driving to be your remaining distance to California. The road trip ends once you reach California.

Solution:

Potential $s_i = 2360/2^i \geq 0$, satisfying the lower-bound condition, but $s_i - s_{i+1} = 2360/2^{i+1}$ has no positive constant lower bound. Moreover $s_i > 0$ for every finite i , so the trip never reaches California in this idealized process, which means it never halts.

- (6 pts) (b) You initially have 203 coins lying flat on a table, all showing heads. You repeatedly perform the following action: if it is possible to do so, you *must* choose a pair of coins showing

heads and turn over *both* of them to show tails. Otherwise, you may turn over *any one or two coins* of your choice. You continue until no coins are showing heads. Define a time unit to be one “action” that turns over one or two coins, and the potential s_i to be the number of heads showing after i actions.

Solution:

The lower-bound condition holds since $s_i \geq 0$.

However, The decrease condition fails. after 101 forced moves one head remains, and the otherwise rule allows flipping that head with a tail, leaving one head again.

Repeating this action forever is legal, so the process never halt in this case.

- (8 pts) (c) Consider this process for sorting a given array of n distinct integers $A[1, \dots, n]$ in increasing order: repeatedly select any two out-of-order elements (i.e., two elements $A[j]$ and $A[k]$ where $j < k$ but $A[j] > A[k]$) and swap them in the array. The process continues until there are no out-of-order elements, in which case the array is then in sorted order. Define a time unit to be one swap, and the potential s_i after i swaps to be the number of array elements that are *not* in the positions they would occupy if the array were sorted.

Solution:

Use the inversion count $I = |\{(j, k) : j < k, A[j] > A[k]\}|$.

It is nonnegative, and swapping a pair decreases I by at least one.

Thus I satisfies both potential conditions and the process halts after at most I swaps.

8. Euclid’s algorithm, extended.

Given two integers x, y (that are not both zero), one may wonder what values can be obtained by summing integer multiples of x and y , i.e., expressed as $ax + by$ for some (not necessarily positive) integers a, b . It is impossible to obtain any positive integer *smaller* than their GCD $g = \gcd(x, y)$, because $ax + by$ must be divisible by g . But, it turns out that the GCD of x and y *can* always be expressed as $ax + by$ for some integers a, b ; this theorem is known as *Bézout’s identity* (pronounced “BAY-zoo”).

In this problem, you will modify the standard Euclidean algorithm to output not only $g = \gcd(x, y)$ itself, but also a pair (a, b) of integers for which $ax + by = g$. Such integers are called *Bézout coefficients* for x, y . (As we will see later, this is very important for cryptography!) We give most of the algorithm below:

Input: integers $x \geq y \geq 0$, not both zero

Output: a triple (g, a, b) of integers where $g = \gcd(x, y) = ax + by$

```

1: function EXTENDED_EUCLID( $x, y$ )
2:   if  $y = 0$  then
3:     return  $(x, 1, 0)$  // Base case:  $1x + 0y = x = \gcd(x, 0)$ 
4:   else
5:     Divide  $x$  by  $y$ , writing  $x = qy + r$  for integer quotient  $q$  and remainder  $0 \leq r < y$ 
6:      $(g, a', b') \leftarrow \text{EXTENDED\_EUCLID}(y, r)$ 
7:     [FOR YOU TO DETERMINE: compute appropriate  $a, b$ ]
8:     return  $(g, a, b)$ 

```

- (6 pts) (a) State what a and b should be on Line 7, and prove that that $ax + by = g$.
Hint: by recursion/induction, we know that $g = \gcd(y, r)$ and $a'y + b'r = g$.

Solution:

Let $x = qy + r$.

Set $a = b'$ and $b = a' - qb'$.

Then $ax + by = b'(qy + r) + (a' - qb')y = a'y + b'r = g$.

- (8 pts) (b) **Run the Extended Euclid algorithm by hand** to find Bézout coefficients for the input $(x, y) = (376, 203)$, and show that the output is correct. (You may use a calculator/computer only for the division steps.) Fill in the table below with a ‘trace’ of the execution, i.e., all the variables’ values in all the iterations. Also include the potential values $s = x + y$, the ratios (as fractions) of potentials s_{i+1}/s_i for adjacent iterations, and Y/N indications of whether $s_{i+1}/s_i \leq 2/3$.

The entries labeled ‘input’, ‘division’, s , and s_{i+1}/s_i should be filled from top to bottom, corresponding to the recursive calls; the ‘recursive answer’ and ‘output’ entries will need to be filled from bottom to top, corresponding to the ‘post-processing’ (Line 7) of each recursive call’s results. The entries for a, b should match those of a', b' one row above. Put ‘-’ for any entries that are not defined due to the base case.

Solution:

i	input		division		rec ans		output		s	s_{i+1}/s_i	$\leq 2/3 ?$
	x	y	q	r	a'	b'	a	b			
0	376	203	1	173	75	-88	-88	163	579	376/579	Y
1	203	173	1	30	-13	75	75	-88	376	203/376	Y
2	173	30	5	23	10	-13	-13	75	203	53/203	Y
3	30	23	1	7	-3	10	10	-13	53	30/53	Y
4	23	7	3	2	1	-3	-3	10	30	9/30	Y
5	7	2	3	1	0	1	1	-3	9	3/9	Y
6	2	1	2	0	1	0	0	1	3	1/3	Y
7	1	0	-	-	-	-	1	0	1	-	-

The recursive base case is $(g, a, b) = (1, 1, 0)$ for $(1, 0)$. The table records the bottom-up back-substitution. In particular,

$$1 = 75 \cdot 203 - 88 \cdot 376,$$

so the final output is $(g, a, b) = (1, -88, 163)$.